

CS 473 (Spring 2025)
Homework 8 (due Apr 3 Thu 10am)

Instructions: As in previous homeworks.

Problem 7.1: Let $G = (V, E)$ be an unweighted graph with n vertices and m edges ($m \geq n$). A *loop* of length $\ell \geq 2$ refers to a walk $\langle v_1, v_2, \dots, v_\ell, v_1 \rangle$ such that $v_1, v_2, \dots, v_\ell$ are all distinct and $v_1v_2, v_2v_3, \dots, v_\ell v_1 \in E$. Note that a loop is the same as a (simple) cycle, except for the case of $\ell = 2$ (usually $\langle v_1, v_2, v_1 \rangle$ is not considered a cycle, but is a loop of length 2 if $v_1v_2 \in E$). We study the following problem: find a vertex-disjoint collection of loops and paths, maximizing their total length.

- (a) (20 pts) Show that there always exists an optimal solution using only loops of length 2 or odd lengths, and no paths. (So, we can redefine the problem without paths and the problem is the same.)
- (b) (10 pts) If G is bipartite, show that the optimal total length is the same as $2k^*$, where k^* is the maximum matching size. (So, the problem is equivalent to maximum matching in the bipartite case.)
- (c) (70 pts) Now consider the problem in the general case when G is not bipartite. Show that the problem can also be reduced to bipartite maximum matching and so can be solved in $O(mn)$ time by the Hungarian method or $O(m\sqrt{n})$ time by Hopcroft and Karp's algorithm.
(Hint: Construct a new bipartite graph G' where we create two versions of each vertex and two versions of each edge... Carefully prove correctness.)

Solution:**(a) Even-Length Loops ($\ell \geq 4$):**

Suppose an optimal solution contains an even-length loop $C = \langle v_1, v_2, \dots, v_{2k}, v_1 \rangle$ where $k \geq 2$. Since the vertices are distinct, we can decompose C into k disjoint 2-loops:

$$\langle v_1, v_2, v_1 \rangle, \langle v_3, v_4, v_3 \rangle, \dots, \langle v_{2k-1}, v_{2k}, v_{2k-1} \rangle.$$

The original loop has $2k$ edges, and the decomposed loops also contribute exactly $2k$ edges. Thus, replacing C with these 2-loops preserves optimality while eliminating even-length loops ($\ell \geq 4$).

Paths:

Assume an optimal solution contains a path $P = \langle v_1, v_2, \dots, v_{k+1} \rangle$ with k edges.

- If v_1 and v_{k+1} are adjacent, we can form a loop of length $k + 1$, increasing the total edge count from k to $k + 1$. This contradicts the optimality of the original solution, so such a path cannot exist in an optimal solution.
- If v_1 and v_{k+1} are not adjacent, consider the number of vertices $k + 1$:
 - If $k + 1$ is even, we can pair the vertices into $\frac{k+1}{2}$ 2-loops, contributing $k + 1$ edges, which is better than the original k edges thus cannot exist.
 - If $k + 1$ is odd, we can leave v_{k+1} unused and construct $\frac{k}{2}$ 2-loops, which does not decrease total length k and preserves optimality.

Thus, even-length loops and paths can always be replaced by 2-loops without decreasing the total length, so they do not appear in an optimal solution.

- (b) Since the vertices of a bipartite graph can be partitioned into two independent sets A and B , every edge connects A and B . If an odd-length cycle existed, its start and end vertices would have to belong to the same set, which is not allowed in bipartite graph. Therefore, all cycles in a bipartite graph must have even lengths. According to part (a), we can conclude that the optimal solution consists solely of vertex-disjoint 2-loops.

Each 2-loop corresponds to a matching edge, contributing a length of 2. A collection of vertex-disjoint loops corresponds to a matching M , where the number of edges is $|M|$, and the total length is $2|M|$. Thus, the maximum matching k^* yields a total length of $2k^*$, which is the theoretical maximum.

Assume there exists a solution with a total length $L > 2k^*$. Then, the corresponding matching size would be $|M| = L/2 > k^*$, contradicting the fact that k^* is the maximum matching size. Therefore, the optimal total length must be $2k^*$.

(c) Construct the Bipartite Graph G' :

Split each vertex $v \in V$ into two copies v^L and v^R , placing them in separate partitions A and B of G' so that $A = \{v^L \mid v \in V\}$ and $B = \{v^R \mid v \in V\}$.

For each edge $(u, v) \in E$, add two edges in G' : (u^L, v^R) and (v^L, u^R) .

An odd-length $2k + 1$ cycle in G

$$v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow \dots \rightarrow v_{2k+1} \rightarrow v_1$$

forms a matching of size $2k + 1$ in G' :

$$v_1^L v_2^R, v_2^L v_3^R, v_3^L v_4^R, \dots, v_{2k+1}^L v_1^R.$$

A 2-loop $\langle u, v, u \rangle$ corresponds to selecting both $u^L v^R$ and $v^L u^R$ in G' .

Proof of Correctness:

i. From G to G' :

Every vertex-disjoint collection of loops in G (length-2 or odd) maps to a matching in G'

For a loop (length-2 or odd), match points alternately in G' , as stated above.

The total length in G is the size of the matching in G' .

ii. From G' to G :

Every matching in G' maps back to a vertex-disjoint collection of loops in G :

A matching of alternating edges

$$v_1^L v_2^R, v_2^L v_3^R, v_3^L v_4^R, \dots, v_n^L v_1^R$$

corresponds to a loop

$$v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow \dots \rightarrow v_n \rightarrow v_1$$

in G . From part (a), this is equivalent to a collection of vertex-disjoint 2-loops and odd-length loops.

A matching

$$v_1^L v_2^R, v_2^L v_3^R, v_3^L v_4^R, \dots, v_{n-1}^L v_n^R$$

corresponds to a path

$$v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow \dots \rightarrow v_n$$

in G . If such a matching exists in the maximum matching of G' , it implies that the vertices v_n^L and v_1^R are unmatched. However, by the construction of G' , there must exist an edge $v_n^L v_1^R$, which could have been matched, resulting in a loop in G . Hence, path cannot exist in G' 's maximum matching.

The property of matching guarantees that no vertex is used twice thus G is vertex-disjoint.

Therefore, the maximum matching of G' yields the maximum total length of G . G' has $2n$ vertices and $2m$ edges, which can be solved in $O(mn)$ time by the Hungarian method or $O(m\sqrt{n})$ time by Hopcroft and Karp's algorithm.

Problem 7.2: Given an undirected graph $G = (V, E)$ and an integer d , we want determine whether it is possible to direct the edges so that the resulting directed graph has maximum **out-degree** at most d . Describe how to solve this problem by reduction to maximum flow. Prove correctness of your method.

Example: for the undirected graph below (left) and $d = 2$, the answer is yes, and one solution is shown on the right (there are many other solutions).



(Hint: start with a bipartite graph where the vertices on the left side are *edges* in G and the vertices on the right side are the vertices in G ; add a source and a sink...)

Solution:

Construct the Bipartite Graph:

1. On the left side, create one node e for each edge $e \in E$ of the original graph G .
2. On the right side, create one node v for each vertex $v \in V$ of the original graph G .
3. Add a source node s and a sink node t .
4. For each edge node e on the left, add an arc $(s \rightarrow e)$ of capacity 1, representing the need to pick exactly one direction for that edge.
5. Suppose the edge e in G connects vertices u and v , add arcs $(e \rightarrow u)$ and $(e \rightarrow v)$, each with capacity 1. This enforces that the 1 unit of flow from edge e can be sent either from vertex u or from vertex v , but not both simultaneously (by the integrality theorem).
6. For each vertex v on the right side, add an arc $(v \rightarrow t)$ of capacity d . The capacity d enforces that at most d edges can flow **out** of vertex v .
7. Compute the maximum flow using Ford-Fulkerson.
8. If there is a flow of value $|E|$, that means we managed to push 1 unit of flow for every edge node to exactly one of its two endpoints, without exceeding the capacity d at any vertex. Hence we get a valid orientation with out-degree $\leq d$.
9. If the max flow is less than $|E|$, then no assignment of directions can keep every vertex's out-degree $\leq d$.

Proof of Correctness:

1. If there is an orientation of G with out-degree $\leq d$, we can construct a flow of value $|E|$:
 - For each edge $e = \{u, v\}$, suppose it is oriented $u \rightarrow v$. Then send 1 unit of flow from s to edge node e , and then from e to the vertex u in the flow network.
 - Because each vertex v has at most d edges oriented out of it in the original orientation, it receives flow from at most d of its edges. Hence the flow into v is at most d , so the capacity $(v \rightarrow t)$ is never exceeded.
 - Thus, we achieve an integer flow of size exactly $|E|$.
2. If the max flow in the constructed network is $|E|$, then each edge node e sends exactly 1 unit of flow to exactly one of its endpoints in G . That determines a direction of e . Since no vertex node v can exceed capacity d to the sink, at most d edges are directed out of any vertex v .

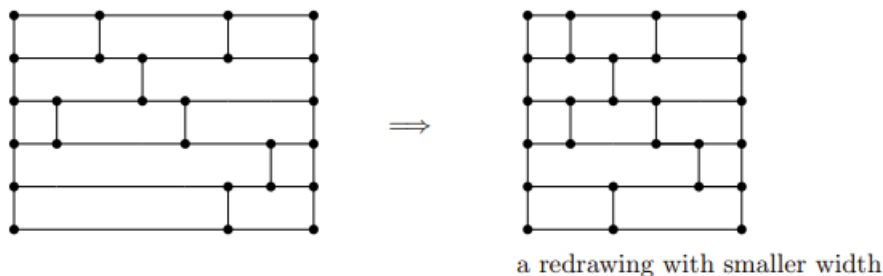
Hence the flow network construction is both sound (no invalid orientations can achieve full flow) and complete (every valid orientation induces a full flow).

Problem 7.3: We are given a set of n axis-aligned rectangles $R = \{r_1, \dots, r_n\}$, where each rectangle r_i has a cost a_i . The rectangles r_i all have height 1 and have integer x - and y -coordinates, and no two rectangles intersect except along the boundaries, and their union is a rectangle $U(R)$. We want to redraw the rectangles as $R' = \{r'_1, \dots, r'_n\}$, whose union is a new rectangle $U(R')$, so that the rectangles r'_i still have height 1 and have integer x - and y -coordinates, and the adjacency structure is unchanged, i.e., if a side of the rectangle r_i touches a side of rectangle r_j , then the corresponding side of r'_i touches the corresponding side of r'_j .

Describe a polynomial-time algorithm to compute a redrawing S' that minimizes $\sum_{i=1}^n a_i \cdot \text{width}(r'_i)$, where $\text{width}(r'_i)$ denotes the width of r'_i .

You may assume that there is a polynomial-time algorithm for the following version of the *minimum-cost flow* problem: given a directed graph $G = (V, E)$ with source s and sink t and a value d , where each edge $e \in E$ is given numbers $\ell(e), c(e), \text{cost}(e)$, compute a flow f of value d , minimizing $\sum_{e \in E} \text{cost}(e) \cdot f(e)$, such that for every $e \in E$, we have $\ell(e) \leq f(e) \leq c(e)$, and we have conservation of flow at every vertex in $V - \{s, t\}$.

(Hint: create a vertex for each rectangle...)



Solution:

Algorithm 1 Optimal Rectangle Redrawing using Minimum-Cost Flow

Require: A set of rectangles $R = \{r_1, r_2, \dots, r_n\}$, each with height 1, integer coordinates, and cost a_i .

Ensure: A redrawing R' that minimizes $\sum_{i=1}^n a_i \cdot \text{width}(r'_i)$.

```
1: for each rectangle  $r_i$  in  $R$  do
2:    $\lfloor$  Add a vertex  $v_i$  to  $V$ .
3: Add source  $s$  and sink  $t$  to  $V$ .
4: for each rectangle  $r_i$  in the first row do
5:    $\lfloor$  Add edge  $(s, v_i)$  to  $E$ .
6: for each row in  $R$  except the last row do
7:   for each rectangle  $r_i$  in the row do
8:     for each rectangle  $r_j$  in the next row do
9:       if  $r_i$  and  $r_j$  share a side then
10:       $\lfloor$   $\lfloor$  Add edge  $(v_i, v_j)$  to  $E$ .
11: for each rectangle  $r_i$  in the last row do
12:    $\lfloor$  Add edge  $(v_i, t)$  to  $E$ .
13: for each rectangle  $r_i$  do
14:    $\lfloor$  Set lower bound  $\ell(v_i) = 1$ .
15:    $\lfloor$  Set upper bound  $c(v_i) = \text{width}(r_i)$ .
16:    $\lfloor$  Set cost  $\text{cost}(v_i) = a_i$ .
17: Set demand  $d$  to the total width of  $U(R)$ .
18: Compute flow  $f$  minimizing  $\sum_{i=1}^n a_i \cdot \text{width}(r'_i)$ .
19: Redraw each rectangle with  $\text{width}(r'_i)$ .
20: return Redrawing  $S'$ .
```

Proof of correctness:

- Since we have conservation of flow at every vertex in $V - \{s, t\}$, the redrawing S' remains a rectangle.
- Each rectangle width remains within valid bounds, i.e., $1 \leq \text{width}(r'_i) \leq \text{width}(r_i)$
- Step 7 ~ 11 guarantees that the new layout maintains the adjacency structure.

Creating the directed graph takes polynomial time, computing the minimum-cost flow takes polynomial time. Thus, the overall algorithm runs in polynomial time.